



BlockingQueue 제대로 파헤치기 (2)

메시지 브로커(MMMQ) 개발기 3

모코

목차

- 배경
- 실험 환경
- 실험 진행
- 실험 결과
- 결과 분석
- 결론

목차

- 배경
- 실험 환경
- 실험 진행
- **실험 결과**
- **결과 분석**
- **결론**

들어가기 앞서

ABQ

- 단일 락 구조
- 배열 기반

LBQ

- 삽입/삭제 락 분리 구조
- 연결 리스트 기반

실험 결과

실험 결과

2 core 50 Thread

큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)	생산 비중	소비 비중	캐시 미스 횟수 (회/ms)	IPC	xsnp_hitm
10%	ABQ_1	1,133.75 	58.72%	41.28% 	18,145.96	1.1 	0.1702 
10%	LBQ_1	5,623.38 	49.99%	50.01%	39,414.97	0.48 	0.7795 
10%	ABQ_50	20,306.64 	50.09%	49.91%	17,278.44 	1.19 	0.008 
10%	LBQ_50	7,355.29 	50.82%	49.18%	22,880.82 	0.53 	0.1791 
100%	ABQ_1	3,161.07 	81.58%	18.42% 	19,529.81	1.37 	0.0869 
100%	LBQ_1	4,449.90 	49.98%	50.02%	51,394.25	0.53 	0.5031 
100%	ABQ_50	20,359.63 	50.00%	50.00%	15,246.14 	1.21 	0.005 
100%	LBQ_50	7,394.79 	52.56%	47.44%	23,991.66 	0.46 	0.1388 

실험 결과

2 core 100 Thread

큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)	생산 비중	소비 비중	캐시 미스 횟수 (회/ms)	IPC	xsnp_hitm
10%	ABQ_1	747.20 	63.08%	36.92% 	1,378.99	0.99 	0.0109 
10%	LBQ_1	4,034.09 	49.96%	50.04%	5,076.07	0.56 	0.112 
10%	ABQ_100	19,456.23 	49.97%	50.03%	3,203.99 	1.33 	0.0008 
10%	LBQ_100	7,214.39 	50.34%	49.66%	4,626.92 	0.75 	0.0237 
100%	ABQ_1	2,458.97 	90.56%	9.44% 	1,695.51	1.29 	0.0093 
100%	LBQ_1	4,230.01 	49.95%	50.05%	7,644.61	0.59 	0.1989 
100%	ABQ_100	19,582.77 	49.98%	50.02%	1,638.78 	1.46 	0.0006 
100%	LBQ_100	6,845.90 	54.44%	45.56%	5,278.40 	0.65 	0.024 

실험 결과

4 core 50 Thread

큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)	생산 비중	소비 비중	캐시 미스 횟수 (회/ms)	IPC	xsnp_hitm
10%	ABQ_1	979.96 	60.05%	39.95% 	15,461.96	1.06 	0.1828 
10%	LBQ_1	4,025.65 	49.99%	50.01%	51,122.74	0.47 	0.4339 
10%	ABQ_50	20,519.79 	50.01%	49.99%	12,682.80 	1.31 	0.0079 
10%	LBQ_50	7,552.56 	50.02%	49.98%	16,300.24 	0.58 	0.2349 
100%	ABQ_1	2,873.46 	84.74%	15.26% 	19,052.35	1.14 	0.1293 
100%	LBQ_1	4,692.69 	49.99%	50.01%	59,013.95	0.43 	1.0053 
100%	ABQ_50	18,825.69 	49.98%	50.02%	8,246.39 	1.30 	0.0053 
100%	LBQ_50	8,283.68 	50.20%	49.80%	30,608.68 	0.50 	0.1963 

실험 결과

4 core 100 Thread

큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)	생산 비중	소비 비중	캐시 미스 횟수 (회/ms)	IPC	xsnp_hitm
10%	ABQ_1	637.02 	65.32%	34.68% 	483.22	1.16 	0.0074 
10%	LBQ_1	4,247.57 	49.96%	50.04%	4,630.62	0.69 	0.085 
10%	ABQ_100	19,641.51 	50.00%	50.00%	1,693.86 	1.41 	0.0007 
10%	LBQ_100	6,382.36 	50.43%	49.57%	3,283.32 	0.80 	0.0273 
100%	ABQ_1	2,565.45 	88.85%	11.15% 	2,225.25	1.35 	0.0098 
100%	LBQ_1	4,007.78 	49.95%	50.05%	7,226.02	0.61 	0.058 
100%	ABQ_100	19,257.95 	50.01%	49.99%	1,663.71 	1.48 	0.0006 
100%	LBQ_100	7,627.44 	51.68%	48.32%	4,760.17 	0.83 	0.0212 

결과 분석

결과 분석

MPSC(Multi-Producer Single-Consumer) 환경

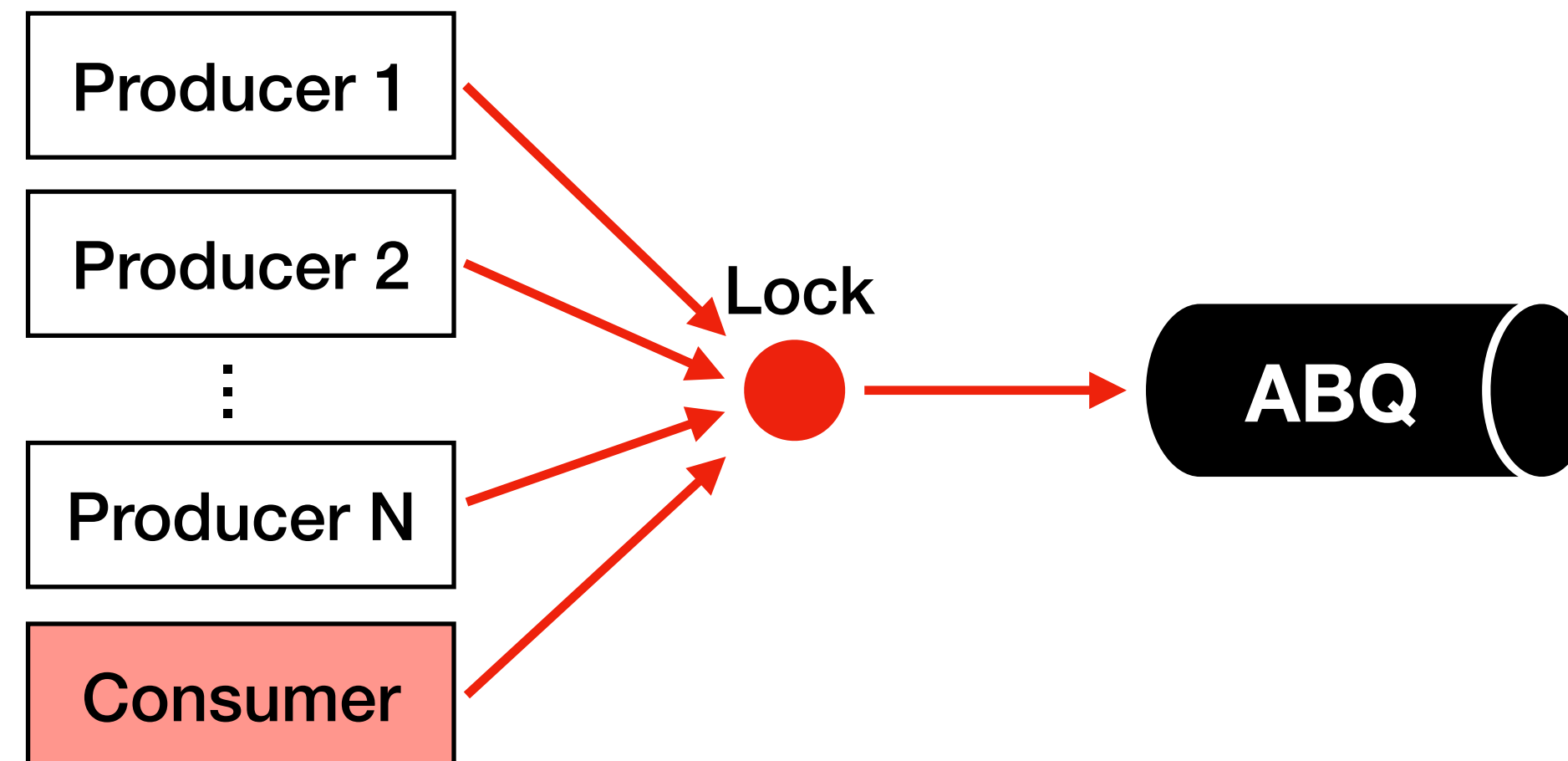
MPSC 환경

ABQ의 병목: 단일 락 경합으로 인한 소비 지연과 생산 실패

- ABQ는 단일 ReentrantLock을 기반으로 생산/소비를 제어함
- 단일 소비자 스레드는 여러 생산자 스레드들과 락 경합을 벌여야 함
-> 소비자의 락 획득 확률이 생산자 집단 대비 크게 낮아짐

$$\text{생산자의 락 획득 확률} = \frac{\text{생산자 스레드 수}}{\text{생산자 스레드 수} + 1}$$

$$\text{소비자의 락 획득 확률} = \frac{1}{\text{생산자 스레드 수} + 1}$$



MPSC 환경

ABQ의 병목: 단일 락 경합으로 인한 소비 지연과 생산 실패

- 생산자 스레드 수 증가 -> 생산자의 락 획득 확률 증가 -> 소비 비중 감소 (■ 참고)
- 큐 용량 증가 -> 더 많은 메시지 수용 가능 -> 큐 포화까지 걸리는 시간 증가 -> 소비 비중 감소 (■ 참고)

코어 수	측정 대상(_n은 소비자)	생산자 스레드 수	큐 용량	소비 비중
2	ABQ_1	50	10%	41.28% ■
2	ABQ_1	50	100%	18.42% ■
2	ABQ_1	100	10%	36.92% ■
2	ABQ_1	100	100%	9.44% ■
4	ABQ_1	50	10%	39.95% ■
4	ABQ_1	50	100%	15.26% ■
4	ABQ_1	100	10%	34.68% ■
4	ABQ_1	100	100%	11.15% ■

MPSC 환경

ABQ의 병목: 단일 락 경합으로 인한 소비 지연과 생산 실패



생산자는 락을 쥐더라도 데이터를 삽입하지 못하여
CPU 사이클만 낭비하게 됨

MPSC 환경

ABQ의 병목: 단일 락 경합으로 인한 소비 지연과 생산 실패

- 일반적인 경우: IPC가 높으면 코어 사용률이 높아서 성능이 높다
- MPSC ABQ의 경우: IPC(참고)는 높지만 생산자의 생산 실패 연산이 많은 부분을 차지해서 성능이 낮다

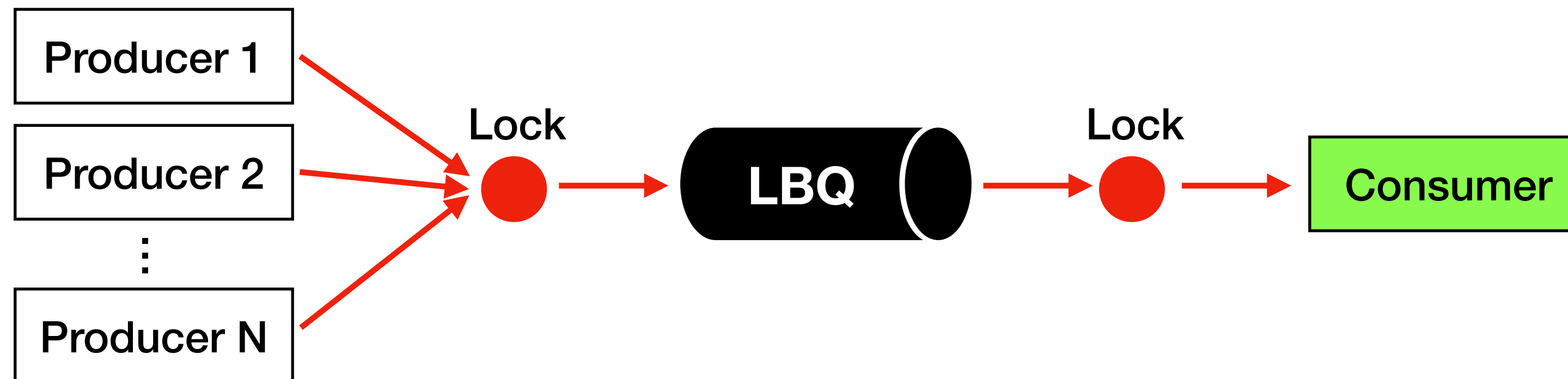
코어, 스레드	큐 용량	측정 대상	IPC	처리량 (ops/ms)	코어, 스레드	큐 용량	측정 대상	IPC	처리량 (ops/ms)	코어, 스레드	큐 용량	측정 대상	IPC	처리량 (ops/ms)	코어, 스레드	큐 용량	측정 대상	IPC	처리량 (ops/ms)
2, 50	10%	ABQ_1	1.1	1,133.75	2, 50	10%	ABQ_1	0.99	747.20	2, 50	10%	ABQ_1	1.06	979.96	2, 50	10%	ABQ_1	1.16	637.02
2, 50	10%	LBQ_1	0.48	5,623.38	2, 50	10%	LBQ_1	0.56	4,034.09	2, 50	10%	LBQ_1	0.47	4,025.65	2, 50	10%	LBQ_1	0.69	4,247.57
2, 50	100%	ABQ_1	1.37	3,161.07	2, 50	100%	ABQ_1	1.29	2,458.97	2, 50	100%	ABQ_1	1.14	2,873.46	2, 50	100%	ABQ_1	1.35	2,565.45
2, 50	100%	LBQ_1	0.53	4,449.90	2, 50	100%	LBQ_1	0.59	4,230.01	2, 50	100%	LBQ_1	0.43	4,692.69	2, 50	100%	LBQ_1	0.61	4,007.78

* IPC(Instructions Per Cycle): CPU가 1개의 클럭 사이클 동안 실행을 완료하는 명령어 수

MPSC 환경

LBQ의 우수: 분리된 락을 통한 소비자 연산 보장

- LBQ는 두 개의 ReentrantLock을 기반으로 생산/소비를 각각 제어함
 - 단일 소비자는 소비 락을 독점하여 데이터를 지속적으로 소비, 큐의 빈 공간 확보
 - 생산자들은 (큐 포화로 인한 생산 실패 없이) 꾸준히 생산 진행



MPSC 환경

결론

- LBQ가 더 높은 처리량 기록 (■ 참고)
- LBQ의 락 분리 구조 -> 소비자가 독립적으로 락 획득 -> 큐를 제때 비워냄

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)
2, 50	10%	ABQ_1	1,133.75 ■
2, 50	10%	LBQ_1	5,623.38 ■
2, 50	100%	ABQ_1	3,161.07 ■
2, 50	100%	LBQ_1	4,449.90 ■

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)
2, 100	10%	ABQ_1	747.20 ■
2, 100	10%	LBQ_1	4,034.09 ■
2, 100	100%	ABQ_1	2,458.97 ■
2, 100	100%	LBQ_1	4,230.01 ■

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)
4, 50	10%	ABQ_1	979.96 ■
4, 50	10%	LBQ_1	4,025.65 ■
4, 50	100%	ABQ_1	2,873.46 ■
4, 50	100%	LBQ_1	4,692.69 ■

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)
4, 100	10%	ABQ_1	637.02 ■
4, 100	10%	LBQ_1	4,247.57 ■
4, 100	100%	ABQ_1	2,565.45 ■
4, 100	100%	LBQ_1	4,007.78 ■

결과 분석

MPMC(Multi-Producer Multi-Consumer) 환경

MPMC 환경

LBQ의 병목 1: 공유 카운터로 인한 하드웨어 레벨 병목

- LBQ에서 count 변수는 양측 스레드가 공유
 - AtomicInteger로 구현되어 있으며, 여러 스레드가 동시에 증감시킴
 - 값 변경을 위한 CAS, 락 획득 전 빠른 실패를 위한 get()
 - 내부적으로 volatile로 관리됨

```
public class LinkedBlockingQueue<E> {  
  
    /** Current number of elements */  
    private final AtomicInteger count = new AtomicInteger();  
}
```

* CAS(Compare And Swap): 특정 메모리 주소의 값이 예상한 기존 값과 일치하는 경우에만 새로운 값으로 덮어쓰는 원자적 하드웨어 명령

* volatile: 여러 스레드가 접근하는 변수의 값을 항상 메인 메모리 기준으로 읽고 쓰도록 하여 가시성과 순서를 보장하는 키워드

MPMC 환경

LBQ의 병목 1: 공유 카운터로 인한 하드웨어 레벨 병목

- LBQ에서 count 변수는 양측 스레드가 공유
 - AtomicInteger로 구현되어 있으며, 여러 스레드가 동시에 증감시킴
 - 값 변경을 위한 CAS, 락 획득 전 빠른 실패를 위한 get()
- 내부적으로 volatile로 관리됨

```
public class LinkedBlockingQueue<E> {  
  
    /** Current number of elements */  
    private final AtomicInteger count = new AtomicInteger();  
}
```

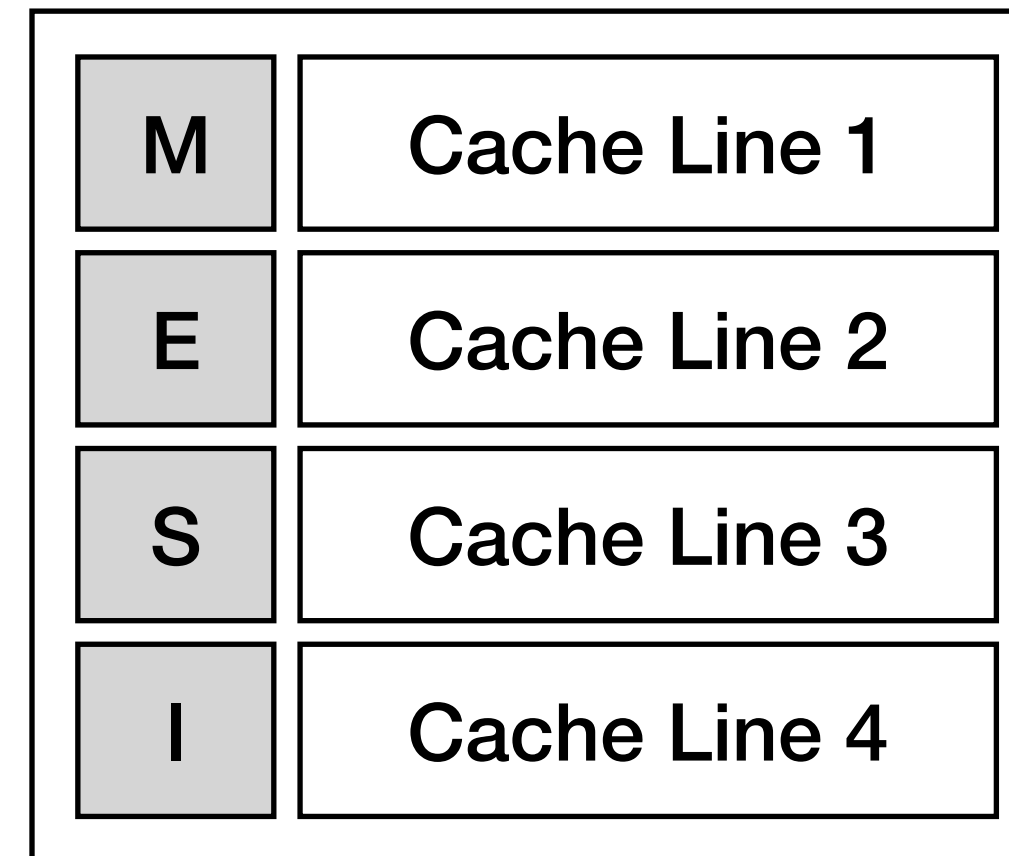
* CAS(Compare And Swap): 특정 메모리 주소의 값이 예상한 기존 값과 일치하는 경우에만 새로운 값으로 덮어쓰는 원자적 하드웨어 명령

* volatile: 여러 스레드가 접근하는 변수의 값을 항상 메인 메모리 기준으로 읽고 쓰도록 하여 가시성과 순서를 보장하는 키워드

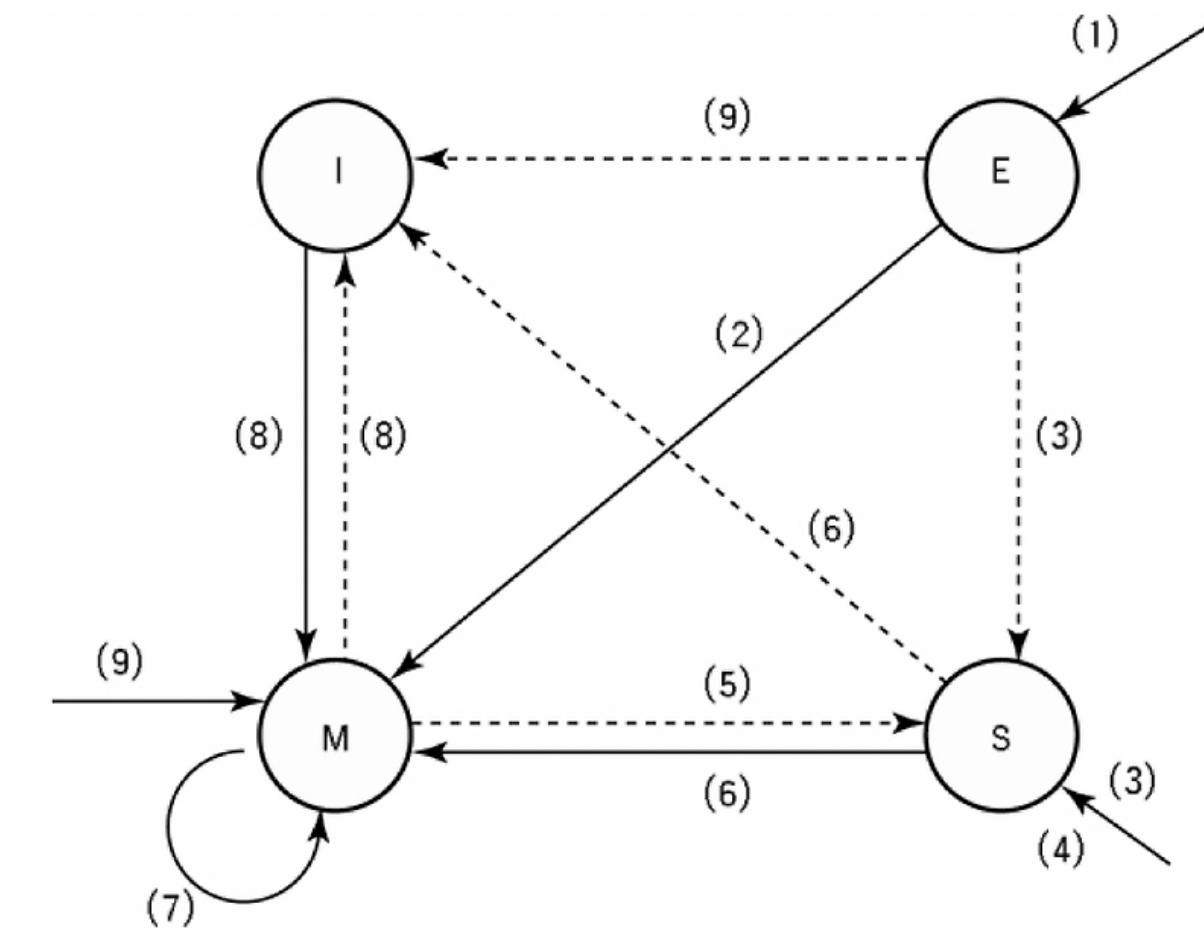
MESI 프로토콜

캐시 일관성 프로토콜

캐시 데이터의 상태



Cache



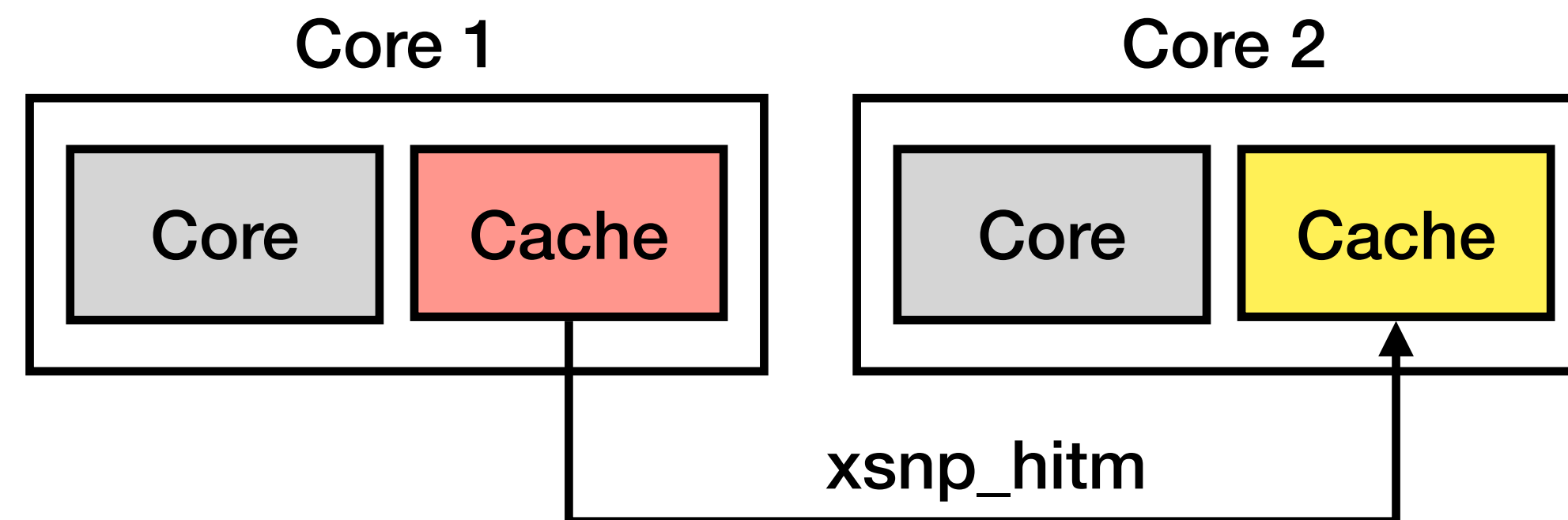
상태 전이도

- 수정 (M : Modified) 상태 : 데이터가 수정(변경) 된 상태
- 베타 (E : Exclusive) 상태 : 유일한 복사본이고, 주기억장치의 내용과 동일한 상태
- 공유 (S : Shared) 상태 : 데이터가 두 개 이상의 프로세서 캐시에 적재되어 있는 상태
- 무효 (I : Invalid) 상태 : 데이터가 다른 프로세서에 의해 수정되어 무효가 된 상태

MPMC 환경

LBQ의 병목 1: 공유 카운터로 인한 하드웨어 레벨 병목

- 여러 코어들의 지속적인 count 수정(volatile 변수)
 - > 캐시 라인이 무효화(Invalid)되어 있을 확률 높음
 - > 다른 코어의 캐시(Modified)를 가져옴(xsnp_hitm)



- * xsnp_hitm(cross-snoop hit modified): Modified 캐시라인을 가진 다른 코어로부터 스누프를 통해 최신 캐시를 가져온 상황
- * 스누프(snoop): 캐시 일관성을 위해 다른 코어의 메모리 요청을 모든 캐시가 감시하는 메커니즘

MPMC 환경

LBQ의 병목 1: 공유 카운터로 인한 하드웨어 레벨 병목

- ABQ보다 xsnp_hitm 높음 (■ 참고)

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	xsnp_hitm
2, 50	10%	ABQ_1	0.1702 ■
2, 50	10%	LBQ_1	0.7795 ■
2, 50	10%	ABQ_50	0.008 ■
2, 50	10%	LBQ_50	0.1791 ■
2, 50	100%	ABQ_1	0.0869 ■
2, 50	100%	LBQ_1	0.5031 ■
2, 50	100%	ABQ_50	0.005 ■
2, 50	100%	LBQ_50	0.1388 ■

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	xsnp_hitm
2, 100	10%	ABQ_1	0.0109 ■
2, 100	10%	LBQ_1	0.112 ■
2, 100	10%	ABQ_100	0.0008 ■
2, 100	10%	LBQ_100	0.0237 ■
2, 100	100%	ABQ_1	0.0093 ■
2, 100	100%	LBQ_1	0.1989 ■
2, 100	100%	ABQ_100	0.0006 ■
2, 100	100%	LBQ_100	0.024 ■

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	xsnp_hitm
4, 50	10%	ABQ_1	0.1828 ■
4, 50	10%	LBQ_1	0.4339 ■
4, 50	10%	ABQ_50	0.0079 ■
4, 50	10%	LBQ_50	0.2349 ■
4, 50	100%	ABQ_1	0.1293 ■
4, 50	100%	LBQ_1	1.0053 ■
4, 50	100%	ABQ_50	0.0053 ■
4, 50	100%	LBQ_50	0.1963 ■

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	xsnp_hitm
4, 100	10%	ABQ_1	0.0074 ■
4, 100	10%	LBQ_1	0.085 ■
4, 100	10%	ABQ_100	0.0007 ■
4, 100	10%	LBQ_100	0.0273 ■
4, 100	100%	ABQ_1	0.0098 ■
4, 100	100%	LBQ_1	0.058 ■
4, 100	100%	ABQ_100	0.0006 ■
4, 100	100%	LBQ_100	0.0212 ■

MPMC 환경

LBQ의 병목 1: 공유 카운터로 인한 하드웨어 레벨 병목

- ABQ보다 xsnp_hitm 높음 (■ 참고)
- 캐시를 조회해올 때까지(xsnp_hitm) CPU Stall 발생 -> IPC 감소 (■ 참고)

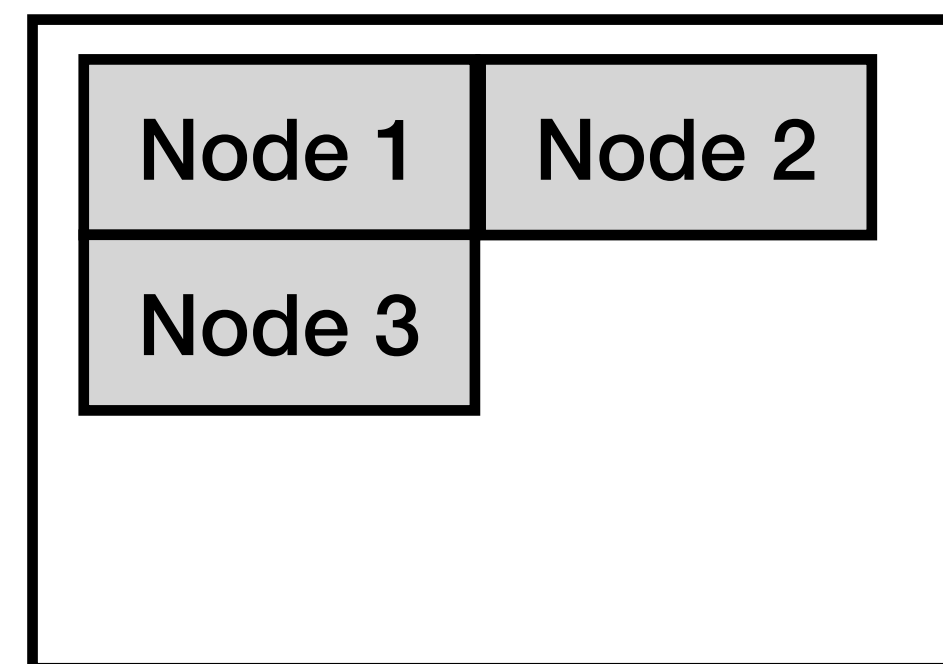
코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	IPC	코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	IPC	코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	IPC	코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	IPC
2, 50	10%	ABQ_50	1.19 ■	2, 100	10%	ABQ_50	1.33 ■	4, 50	10%	ABQ_100	1.31 ■	4, 100	10%	ABQ_100	1.41 ■
2, 50	10%	LBQ_50	0.53 ■	2, 100	10%	LBQ_50	0.75 ■	4, 50	10%	LBQ_100	0.58 ■	4, 100	10%	LBQ_100	0.80 ■
2, 50	100%	ABQ_50	1.21 ■	2, 100	100%	ABQ_50	1.46 ■	4, 50	100%	ABQ_100	1.30 ■	4, 100	100%	ABQ_100	1.48 ■
2, 50	100%	LBQ_50	0.46 ■	2, 100	100%	LBQ_50	0.65 ■	4, 50	100%	LBQ_100	0.50 ■	4, 100	100%	LBQ_100	0.83 ■

* CPU Stall: CPU가 다음 명령어를 실행하지 못하고 대기하는 상태

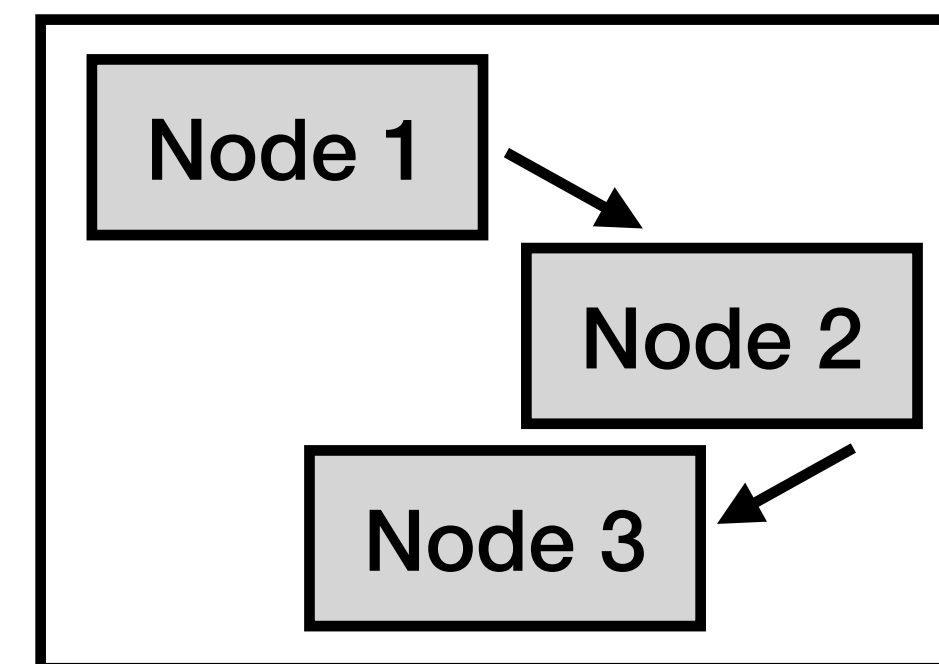
MPMC 환경

LBQ의 병목 2: 동적 객체 할당과 코어 간 캐시 미스

- ABQ는 배열 구조로, 연속된 메모리 공간에 데이터 저장
 - 한 번 읽어온 캐시 라인 내부에 다음 인덱스가 포함되어 있을 확률이 높음 -> 캐시 히트율 높음
- LBQ는 연결 리스트 구조로, 분산된 메모리 공간에 데이터 저장
 - 각 노드에 접근할 때마다 캐시 라인을 새로 읽어와야 함 -> 캐시 히트율 낮음



ABQ



LBQ

MPMC 환경

LBQ의 병목 2: 동적 객체 할당과 코어 간 캐시 미스

- ABQ보다 캐시 미스 횟수가 높음 (■ 참고)

-> CPU Stall -> IPC가 낮음 (■ 참고)

코어, 스레드	큐 용량	측정 대상	캐시 미스 횟수	IPC
2, 50	10%	ABQ_50	17,278 ■	1.19 ■
2, 50	10%	LBQ_50	22,880 ■	0.53 ■
2, 50	100%	ABQ_50	15,246 ■	1.21 ■
2, 50	100%	LBQ_50	23,991 ■	0.46 ■

코어, 스레드	큐 용량	측정 대상	캐시 미스 횟수	IPC
2, 100	10%	ABQ_50	3,203 ■	1.33 ■
2, 100	10%	LBQ_50	4,626 ■	0.75 ■
2, 100	100%	ABQ_50	1,638 ■	1.46 ■
2, 100	100%	LBQ_50	5,278 ■	0.65 ■

코어, 스레드	큐 용량	측정 대상	캐시 미스 횟수	IPC
4, 50	10%	ABQ_100	12,682 ■	1.31 ■
4, 50	10%	LBQ_100	16,300 ■	0.58 ■
4, 50	100%	ABQ_100	8,246 ■	1.30 ■
4, 50	100%	LBQ_100	30,608 ■	0.50 ■

코어, 스레드	큐 용량	측정 대상	캐시 미스 횟수	IPC
4, 100	10%	ABQ_100	1,693 ■	1.41 ■
4, 100	10%	LBQ_100	3,283 ■	0.80 ■
4, 100	100%	ABQ_100	1,663 ■	1.48 ■
4, 100	100%	LBQ_100	4,760 ■	0.83 ■

MPMC 환경

ABQ의 우위 1: 단순한 내부 연산과 객체 생성 없는 참조 매핑

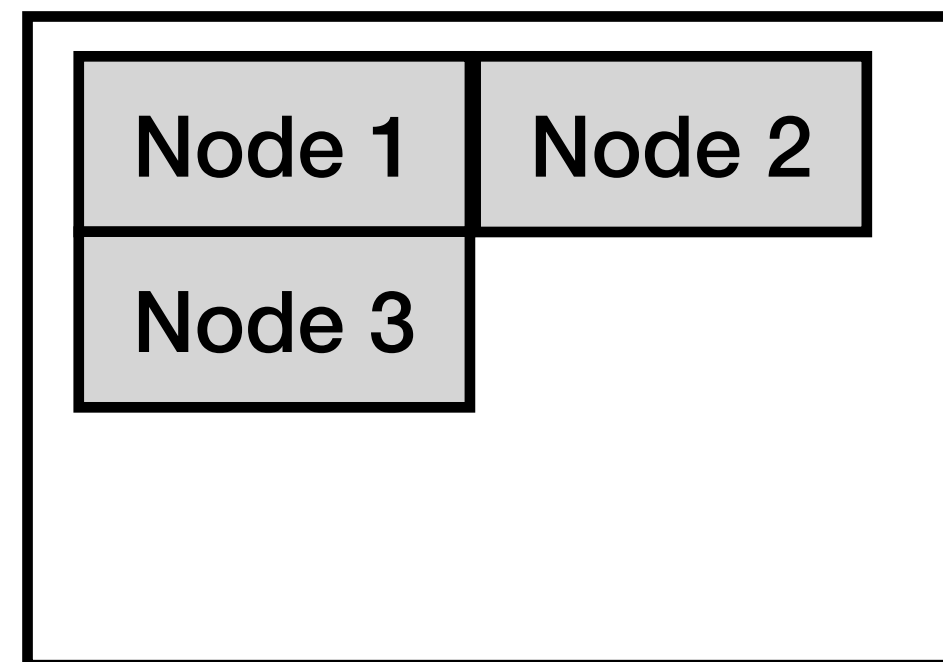
- ABQ는 락 내 동시성이 보장됨
 - count 변수가 AtomicInteger 대신 int 변수로 구현됨
 - 단순 증감 연산이므로 추가적인 오버헤드가 없음

```
public class ArrayBlockingQueue<E> {  
  
    /** Number of elements in the queue */  
    int count;
```

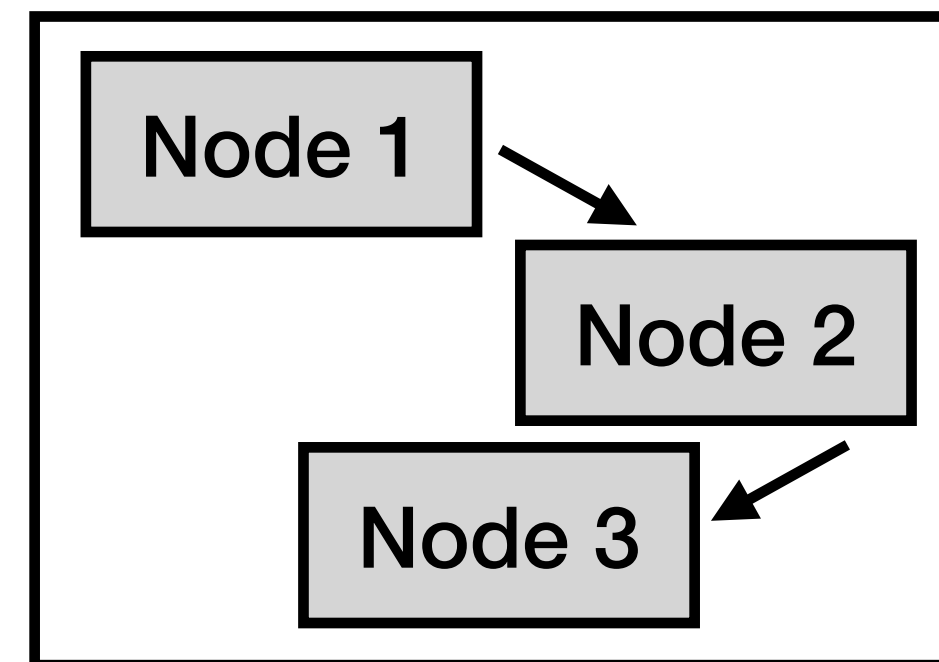

MPMC 환경

ABQ의 우위 1: 단순한 내부 연산과 객체 생성 없는 참조 매핑

- 배열 인덱스를 통해 메시지 객체에 접근함
 - Node 객체로 접근하는 LBQ에 비해 캐시 지역성 우위



ABQ



LBQ

MPMC 환경

ABQ의 우위 1: 단순한 내부 연산과 객체 생성 없는 참조 매핑

- 배열 인덱스를 통해 메시지 객체에 접근함
 - Node 객체로 접근하는 LBQ에 비해 캐시 지역성 우위 -> 캐시 미스 낮음 (■ 참고)

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	캐시 미스 횟수
2, 50	10%	ABQ_50	17,278.44 ■
2, 50	10%	LBQ_50	22,880.82 ■
2, 50	100%	ABQ_50	15,246.14 ■
2, 50	100%	LBQ_50	23,991.66 ■

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	캐시 미스 횟수
2, 100	10%	ABQ_50	3,203.99 ■
2, 100	10%	LBQ_50	4,626.92 ■
2, 100	100%	ABQ_50	1,638.78 ■
2, 100	100%	LBQ_50	5,278.40 ■

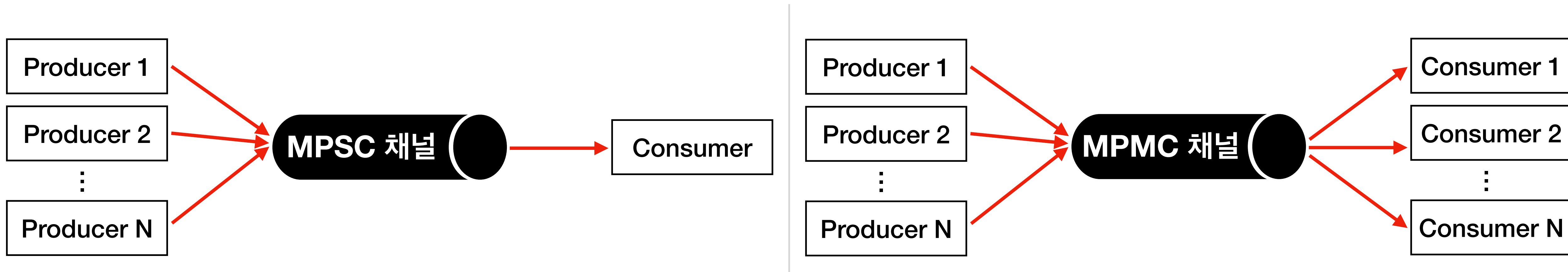
코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	캐시 미스 횟수
4, 50	10%	ABQ_100	12,682.80 ■
4, 50	10%	LBQ_100	16,300.24 ■
4, 50	100%	ABQ_100	8,246.39 ■
4, 50	100%	LBQ_100	30,608.68 ■

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	캐시 미스 횟수
4, 100	10%	ABQ_100	1,693.86 ■
4, 100	10%	LBQ_100	3,283.32 ■
4, 100	100%	ABQ_100	1,663.71 ■
4, 100	100%	LBQ_100	4,760.17 ■

MPMC 환경

ABQ의 우위 2: 생산/소비 균형을 통한 소비 지연 해소

- MPSC 환경 -> 생산자 n vs 소비자 1
 - 소비자가 큐를 제때 비우지 못함 -> 큐가 가득 차면서 생산 실패 -> 코어 점유시간 낭비
- MPMC 환경 -> 생산자 n vs 소비자 n
 - 소비자의 락 획득 빈도 증가 -> 큐의 원활한 소비



MPMC 환경

결론

- ABQ가 더 높은 처리량 기록 (■ 참고)
- LBQ는 공유 카운터 오버헤드와 연결 리스트 구조가 병목
- ABQ는 생산자-소비자 불균형이 해소되어 단일 락 경합으로 인한 소비 지연이 완화됨

코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)	코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)	코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)	코어, 스레드	큐 용량	측정 대상 (_n은 소비자)	처리량 (ops/ms)
2, 50	10%	ABQ_50	20,306.64 ■	2, 100	10%	ABQ_50	19,456.23 ■	4, 50	10%	ABQ_100	20,519.79 ■	4, 100	10%	ABQ_100	19,641.51 ■
2, 50	10%	LBQ_50	7,355.29 ■	2, 100	10%	LBQ_50	7,214.39 ■	4, 50	10%	LBQ_100	7,552.56 ■	4, 100	10%	LBQ_100	6,382.36 ■
2, 50	100%	ABQ_50	20,359.63 ■	2, 100	100%	ABQ_50	19,582.77 ■	4, 50	100%	ABQ_100	18,825.69 ■	4, 100	100%	ABQ_100	19,257.95 ■
2, 50	100%	LBQ_50	7,394.79 ■	2, 100	100%	LBQ_50	6,845.90 ■	4, 50	100%	LBQ_100	8,283.68 ■	4, 100	100%	LBQ_100	7,627.44 ■

결과 분석

MPSC 환경에서의 자료구조별 장단점

MPSC 환경에서의 자료구조별 장단점

ArrayBlockingQueue

장점: 빠르다

- count를 int로 관리하여 내부 연산이 가벼움
- 한 번 읽어온 캐시 라인 내부에 다음 데이터가 포함될 확률이 높아 캐시 히트율이 높음

단점: 불합리하다

- 단일 소비자가 생산자 집단과 경합하여 소비자의 락 획득 확률이 낮음
 - > 소비 지연 -> 큐가 빠르게 포화 상태에 도달 -> 생산까지 연쇄적으로 실패

MPSC 환경에서의 자료구조별 장단점

LinkedBlockingQueue

장점: 합리적이다

- 단일 소비자는 생산자들과 무관하게 소비 락
 - > 소비자 실행 보장 -> 지속적인 소비로 큐의 빈 공간 확보

단점: 느리다

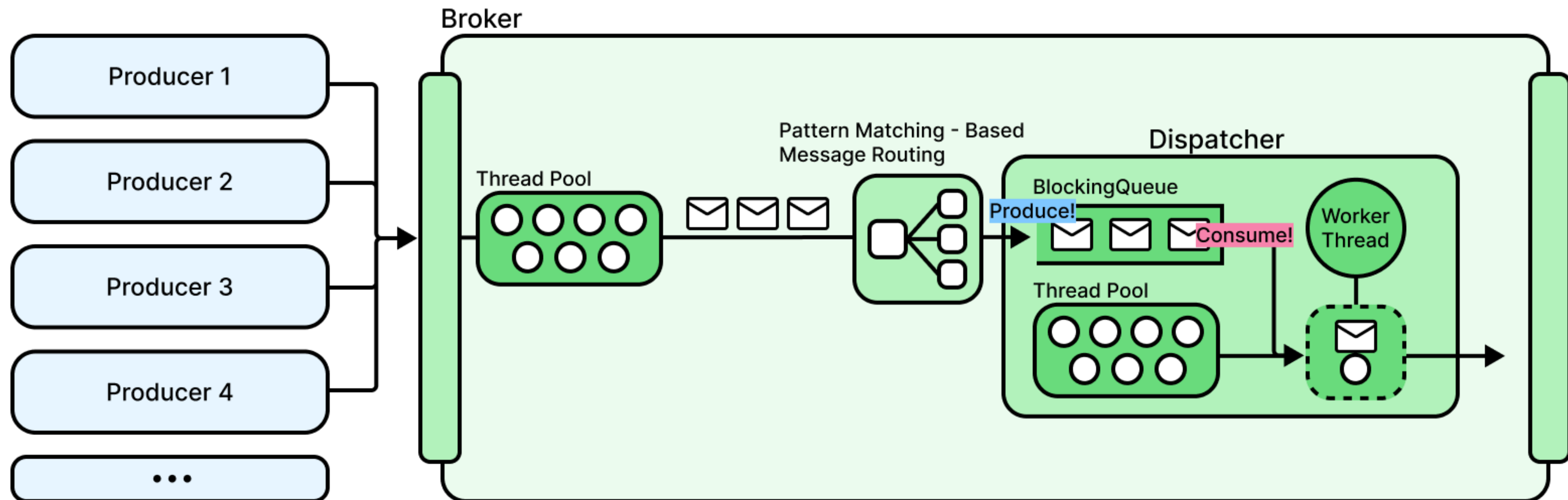
- count 변수를 AtomicInteger로 관리하여 내부 연산이 무거움
- 각 노드에 접근할 때마다 캐시 라인을 새로 읽어와야 함 -> 캐시 히트율 낮음

결론

MMMQ 환경에 적합한 자료구조

MMMQ 환경에 적합한 자료구조

- MMMQ 브로커는 다수의 생산자가 발행한 메시지를 단일 Worker 스레드가 라우팅하는 MPSC 아키텍처



MMMQ 환경에 적합한 자료구조

- LBQ는 비효율적인 요소가 많음 (AtomicInteger volatile/CAS, 연결 리스트 캐시 미스 등)
 - 순수 연산 효율: **ABQ > LBQ**
- 하지만 ABQ는 단일 소비자가 다수의 생산자와 락 획득을 경쟁해야 함.
 - 소비자의 락 획득 확률이 낮아 데이터 소비가 지연됨
 - 락을 얻지 못해 작업을 수행하지 못한다면 ABQ의 하드웨어적 이점은 무의미함
- **LBQ는 락이 분리되어 있어 소비자 실행이 생산자에게 방해받지 않음**

MMMQ 환경에 적합한 자료구조

- 하드웨어적 비효율을 감수하더라도
- 단일 소비자의 실행 기회를 독립적으로 보장하여
- 전체 메시지 처리량을 유지할 수 있는 LBQ를 선택

감사합니다

MMMQ: <https://github.com/moko-meringue/mmmq>

부하테스트: <https://github.com/moko-meringue/blocking-queue-analysis>